Distributed Retrieval–Augmented Generation (RAG) System

Eeshita Deepta, Nithya Rajkumar, Swathy Anand, Swetha Mohan

COMPSCI 532 - Fall 2025

Design Description

Distributed Semantic Search Over Large Text Corpora for RAG Systems

- Built a system that converts a large AG News dataset into searchable vector embeddings using distributed processing
- Supports fast semantic retrieval by indexing document chunks with sharded FAISS
- Allows answering user queries by retrieving the most relevant text passages
- Forms the retrieval backbone for Retrieval-Augmented Generation (RAG) applications

Design Description

System Overview / Data Flow

- Stream and clean raw corpus
- Chunk text into overlapping passages
- Distribute chunks across Spark executors
- Each executor embeds text using a shared SentenceTransformer model
- Store embeddings as Parquet; build FAISS index shards
- Queries run in parallel on shards → merged top-k results

Raw Text → Chunking → Spark Embedding → FAISS Shards → Distributed Query → Results

Design Description

Key Design Decisions

- Used **mapPartitions** so each worker loads the embedding model only once
- Chose **FAISS IndexFlatIP** for fast, training-free, shard-friendly indexing
- Stored all intermediate outputs in **Parquet** for efficient distributed processing

Trade-offs

- CPU-only embedding slower but easier to parallelize
- Chunk overlap improves retrieval quality but increases data size
- Many partitions → more parallelism but slightly higher merge overhead

Accomplishments

- Fully distributed end-to-end pipeline
- Low-latency distributed ANN search
- Successfully scaled on both laptop and GCP cluster

Tests

Sanity Checks

- Queried known facts (e.g., *“What is the capital of France?”*) and verified top-k chunks referenced **Paris**
- Checked that chunking produced **non-empty**, correctly overlapped passages
- Verified embedding vectors were normalized and correct dimension

Correctness Tests

- Compared **distributed ANN top-1** results with **brute-force cosine similarity** on a 2,000-row sample
- High agreement (≈90–100%) confirmed correct FAISS shard search + merge logic
  Verified FAISS local IDs correctly mapped to original text via shard mapping files

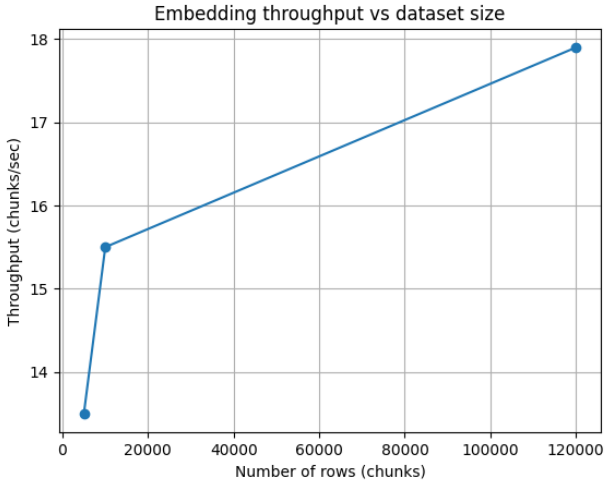
Performance Trends

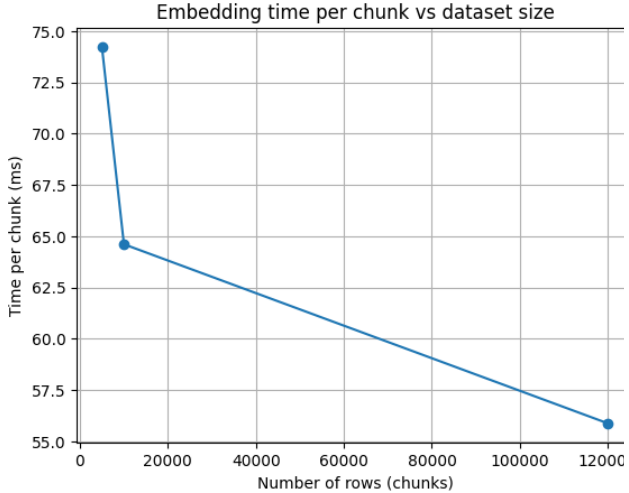
- Larger datasets produced **better embedding throughput** (improved batching)
- Query latency remained consistently **low (<3 ms)** across all scales
- FAISS index building time scaled **linearly** with shard size

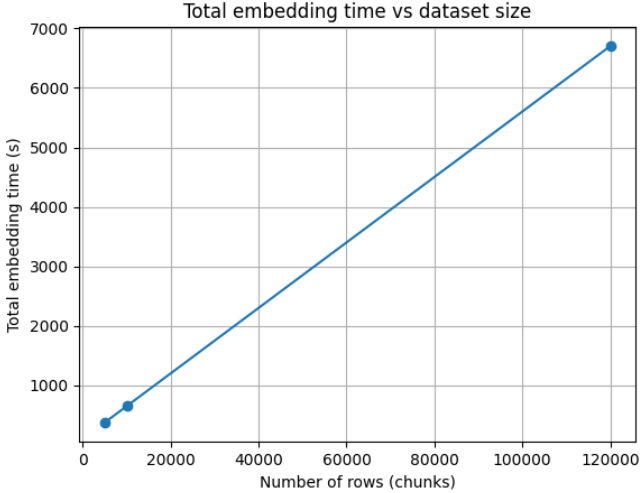
Unit / Component Tests

- Tested partition-level embedding functions (mapPartitions)
- Verified each FAISS shard loads, searches, and returns valid top-k candidates
  Checked manifest and mapping parquet consistency across shards

Experimental Results







Experimental Results

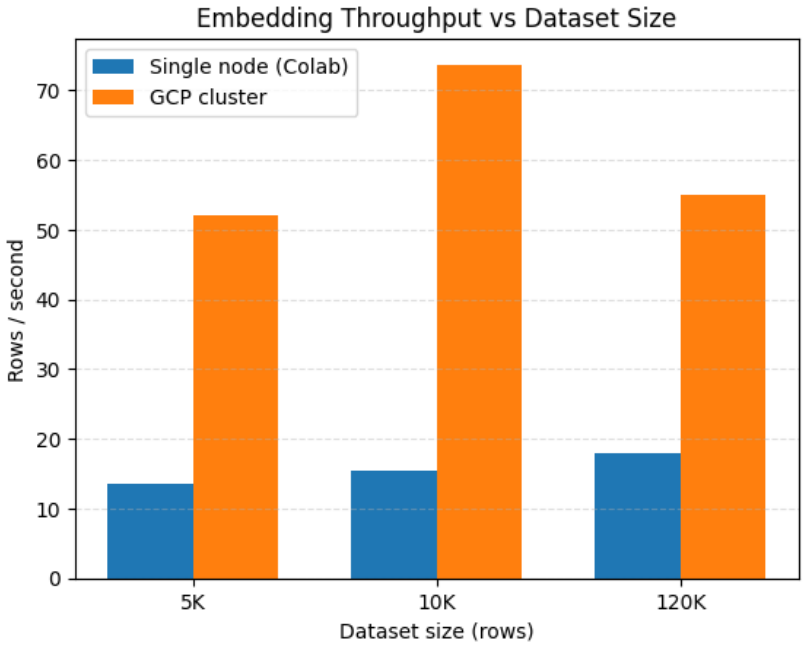
Embedding Performance

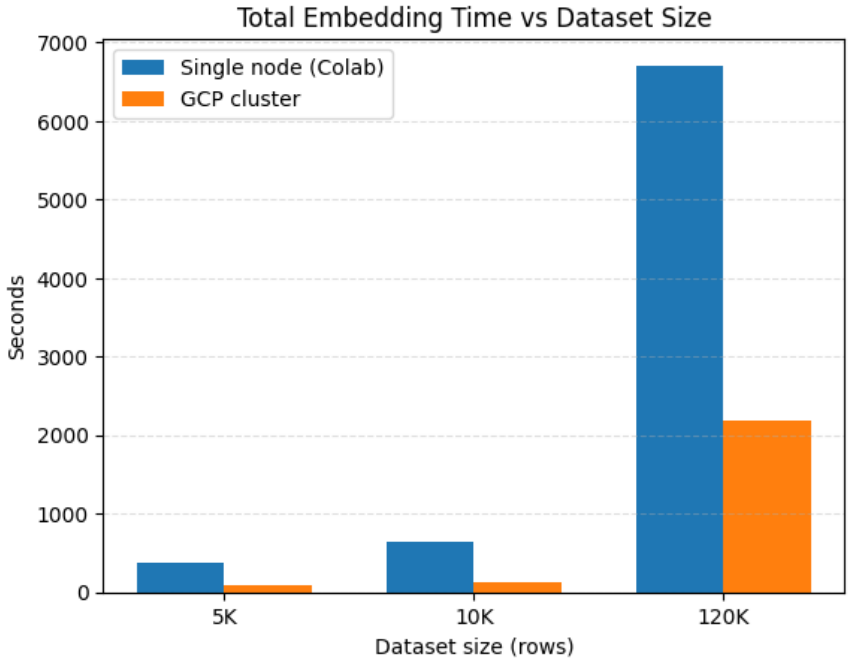
Dataset Size	GCP Cluster – Embedding Time (s)	GCP Throughput (rows/s)	Single Node – Embedding Time (s)	Single Node Throughput (rows/s)
5K	96.0	52.1	371.51	13.5
10K	135.8	73.7	646.59	15.5
120K	2182.7	55.0	6708.42	17.9

FAISS Index Build Time

Dataset Size	GCP FAISS Build Time (s)	Single Node FAISS Build Time (s)	Notes
5K	0.0020	33.94	GCP uses **1 shard**, single-node built **8 shards**
10K	0.0123	6.18	Same: distributed vs multi-shard
120K	0.0273	21.07	GCP extremely fast due to CPU vectorization

Experimental Results





Goals

- COMPLETE — **Data Preparation:** Collected and preprocessed large-scale text corpus in HDFS / Parquet
- COMPLETE — **Distributed Embedding Generation:** Implemented distributed embedding generation using PySpark + PyTorch (mapPartitions) and measure throughput
- COMPLETE— **Index Construction:** Built distributed FAISS index shards and perform parallel ANN retrieval with global top-k merging
- COMPLETE — **Preliminary Benchmarking:** Evaluated system performance (throughput, latency, scalability) and generate visual analysis charts

Incomplete Goals and Possible Improvements

- **POSSIBLE IMPROVEMENTS**
  - **Evaluate different LLMs on top of our retriever**
    Compare end-to-end latency, cost, and answer quality when using multiple LLMs (e.g., GPT vs smaller open-source models) with the same FAISS-based retrieval layer.
  - **Switch to HNSW Indexing for Higher Recall**
    Experiment with **FAISS HNSW** or **IVF+PQ** to achieve faster searches and higher recall, especially at larger scales.
  - **GPU-Accelerated Embedding & Indexing**
    Use GPU-enabled Spark or distributed PyTorch for significantly faster embedding on large corpora.
  - **Move Storage to HDFS or Cloud Buckets**
    Improve scaling by storing input/output in **HDFS** or **GCS**, enabling multiple compute nodes to ingest data in parallel.